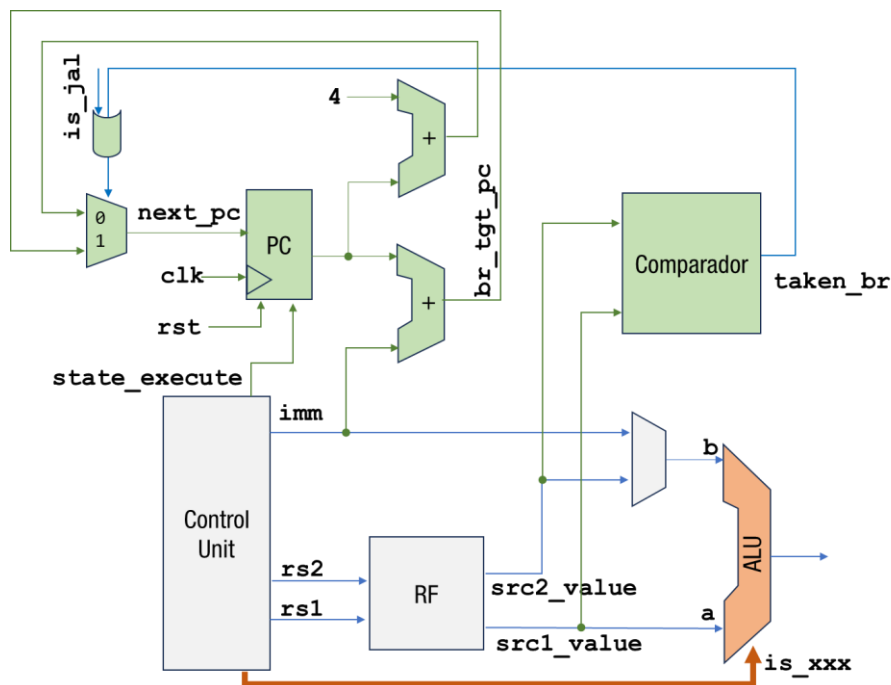




- En el código anterior se cargó el valor inicial de los registros x5 y x6 con el número -100 usando una instrucción de suma, ¿por qué funciona esta instrucción?  
- ¿Cómo implementaría la operación  $x7 = x5 * 4 - x3 / 2$ ? Pruebe su resultado en el simulador.
- ¿Soporta el RV32I multiplicaciones y divisiones? Si no, ¿Cómo se pueden soportar estas operaciones en RISC-V?  

## PARA EL LABORATORIO

De la Figura 12 puede notarse que el conjunto de instrucciones de la ALU que expandiremos en esta guía, excepto LUI y AUIPC, son todas del direccionamiento tipo R e I, cuyo soporte ya fue incluido en el diseño de la ALU en guías anteriores. Esto significa que para implementar estas nuevas instrucciones basta sólo con decodificar la instrucción para generar las señales `is_XXX` que se usarán como entradas de control adicionales a la ALU, y rediseñar la ALU para soportar estas nuevas señales e instrucciones. En la Figura 13 se muestra el diseño que se obtuvo de la guía anterior y la cual soporta las instrucciones tipo R e I. En naranja se resaltan las señales y componentes que deben ser ampliados.



**Figura 13 Implementación actual de nuestro microprocesador RISC-V antes de incluir LUI y AUIPC. Las nuevas señales y bloques que deben modificarse se resaltan en naranja**

Para hacer la implementación se recomienda entonces los siguientes pasos:

1. Decodifique la instrucción para obtener las señales `is_sll` (que se activa para las instrucciones `sll` y `slli`), `is_srl`, `is_sra`, `is_slt`, `is_sltu`, `is_lui`, `is_auipc` (Use el formato indicado en la siguiente tabla). Tenga en cuenta que el bit 5 de `funct7` se usa para diferenciar ciertas instrucciones de desplazamiento.

| funct7     |       |    |    | funct3 |     |    |    | opcode  |         |         |         |  |
|------------|-------|----|----|--------|-----|----|----|---------|---------|---------|---------|--|
| 31         | 25    | 24 | 20 | 19     | 15  | 14 | 12 | 11      | 7       | 6       | 0       |  |
| imm[31:12] |       |    |    |        |     |    |    | rd      | 0110111 |         | U lui   |  |
| imm[31:12] |       |    |    |        |     |    |    | rd      | 0010111 |         | U auipc |  |
| imm[11:0]  |       |    |    | rs1    | 010 |    | rd | 0010011 |         | I slti  |         |  |
| imm[11:0]  |       |    |    | rs1    | 011 |    | rd | 0010011 |         | I sltiu |         |  |
| 0000000    | shamt |    |    | rs1    | 001 |    | rd | 0010011 |         | I slli  |         |  |
| 0000000    | shamt |    |    | rs1    | 101 |    | rd | 0010011 |         | I srli  |         |  |
| 0100000    | shamt |    |    | rs1    | 101 |    | rd | 0010011 |         | I srai  |         |  |
| 0000000    | rs2   |    |    | rs1    | 001 |    | rd | 0110011 |         | R sll   |         |  |
| 0000000    | rs2   |    |    | rs1    | 010 |    | rd | 0110011 |         | R slt   |         |  |
| 0000000    | rs2   |    |    | rs1    | 011 |    | rd | 0110011 |         | R sltu  |         |  |
| 0000000    | rs2   |    |    | rs1    | 101 |    | rd | 0110011 |         | R srl   |         |  |
| 0100000    | rs2   |    |    | rs1    | 101 |    | rd | 0110011 |         | R sra   |         |  |

2. Modifique la entrada de operación de la ALU para soportar estas nuevas señales (excepto `is_lui` & `is_auipc`).
3. Modifique internamente la ALU para soportar las nuevas instrucciones. Hay ciertas instrucciones que requieren un tratamiento especial debido al bit de signo. Estas son `sltu` y `sltiu` (fijar bit si la condición es menor e igual sin signo) y el corrimiento aritmético hacia la derecha (`sra`). Para ello, resulta más fácil crear unas señales que hagan el cálculo previo del resultado antes de pasarlo al MUX que implementa la ALU. Dentro de la implementación de la ALU cree estas señales:

```

1 wire [4:0] shamt;
2 wire [63:0] a_ext;
3 wire [31:0] sltu_result;
4 wire [31:0] slt_result;
5
6 //Valor del shift
7 assign shamt = b[4:0];
8
9 //Extensión de signo de la entrada a para 64bits
10 assign a_ext = { {32{a[31]}}, a };
11
12 //Valor obtenido por set if less than unsigned (sltu)
13 assign sltu_result = {31'b0, a < b};
14
15 //Valor obtenido por set if less than (signed)
16 assign slt_result = (a[31]==b[31]) ? sltu_result : {31'b0, a[31]};

```

La línea 7 extrae de la entrada B de la ALU, el número de bits que será desplazado el valor que ingresa por la entrada A.

La línea 10 extiende el signo del operador A de la ALU en 64 bits. Esto se hace para implementar más fácil la instrucción de desplazamiento aritmético a la derecha.

Las líneas 13 y 16 determinan el valor de las comparaciones, el cual puede ser 1 o 0, donde 1 se genera si el valor del operando A es menor que el operando B.

4. Ahora amplíe el MUX de la ALU para incluir las nuevas instrucciones de la forma:

**SLL:** `a << shamt`  
**SRL:** `a >> shamt`  
**SRA:** `a_ext >> shamt`  
**SLT:** `slt_result`  
**SLTU:** `sltu_result`



Ahora, para implementar **LUI** y **AUIPC**, que son las únicas instrucciones de tipo U, es necesario comprender su funcionamiento:

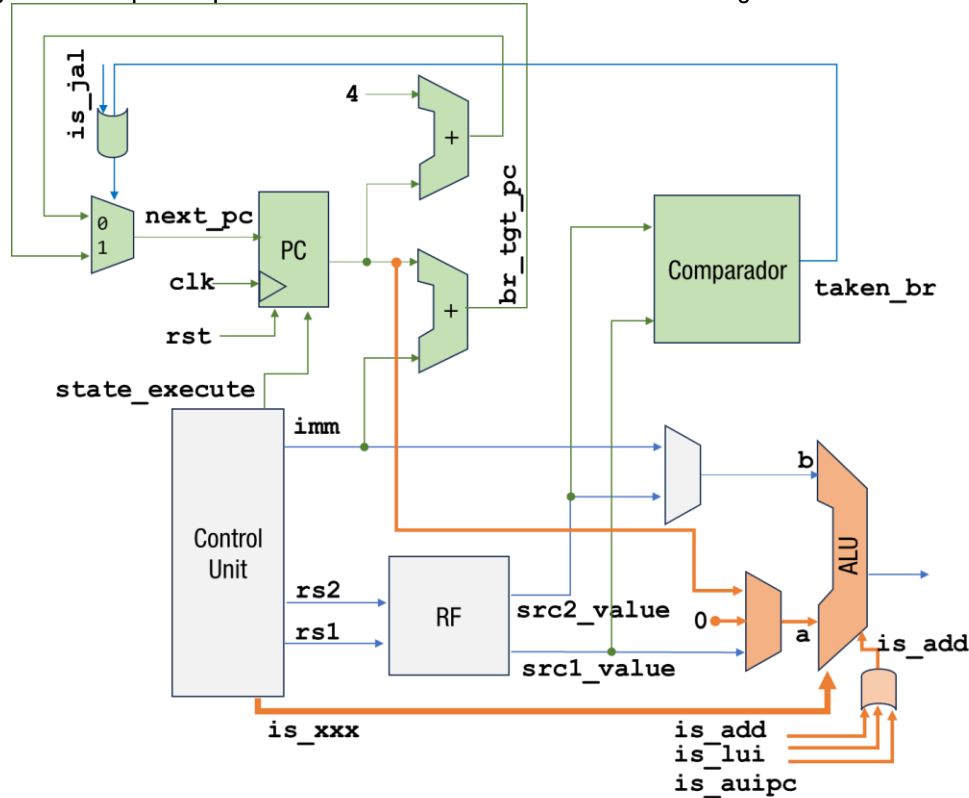
- LUI (Load Upper Immediate): Carga `r_d` con un valor igual al `(imm << 12)`
- AUIPC: Add upper immediate to PC: Carga `r_d` con un valor igual a `(imm << 12) + PC`

Lo anterior significa que, para instrucciones tipo U, la entrada B de la ALU debe ser el dato inmediato desplazado 12 bits a la izquierda, algo similar a como ocurría para las instrucciones tipo I. De hecho, en la Guía 2 ya se consideró este corrimiento para crear el dato inmediato en las instrucciones tipo U, por lo que no debe hacerse ningún cambio en nuestra implementación para incluir este desplazamiento.

Pero para soportar estas instrucciones, la entrada A no puede venir exclusivamente del register file a través del bus `src1_value` como se mostraba en la Figura 13, en su lugar

- Si la instrucción es tipo R o I, la ALU debe usar como entrada A la señal a `src1_value` del register file.
- Si la instrucción es tipo U, la ALU debe hacer una suma.
- Si la instrucción es LUI, la entrada A debe ser 0, para que el resultado de salida de la ALU sea `(imm<12) + 0`.
- Si la instrucción es AUIPC, la entrada A debe ser el PC, para que el resultado de salida de la ALU sea `(imm<12) + PC`.

Estos cambios incluyen un MUX para el parámetro A de la ALU como se muestra en la Figura 14.



**Figura 14 Implementación del microprocesador RISC-V para incluir LUI y AUIPC. LUI y AUIPC hacen una suma entre el inmediato y 0 o PC, respectivamente.**

Una vez termine de ajustar la ALU y la Control Unit, vamos a incluir un programa de prueba. Para hacer el testbench más general usa una IA para haga *refactoring* del código de la ROM usando la directiva `$readmemh`. Nota: `$readmemh` inicializa la memoria con los datos almacenados en un archivo externo. Si hace *refactoring* sobre la ROM de la Guía 3, la IA le sugerirá también los valores que debe poner en un archivo con extensión “.hex”. Estos códigos son los códigos de máquina del programa de la Guía 3. Pruebe el microcontrolador y debe darle exactamente el mismo resultado de dicha Guía.

Ahora, vamos a crear un nuevo programa de pruebas. Puede decirle a la IA que genere un programa de prueba en ensamblador del RV32I para probar las instrucciones sll, srl, ..., o todas las instrucciones de la ALU. No olvide guardar el programa que genera la IA con el nombre **test\_alu.s**

Luego ensamble el programa con el siguiente comando desde el terminal:

```
riscv64-unknown-elf-as -o test_program.elf test_alu.s -march=rv32i
```

Puede ver los códigos de máquina generados usando el siguiente comando en el terminal:

```
riscv64-unknown-elf-objdump test_program.elf -S
```

Usted puede copiar a mano, en el archivo hex, los códigos de máquina mostrados por **objdump**.

Alternativamente puede usar el siguiente comando en el terminal para generar el archivo hex:

```
riscv64-unknown-elf-objcopy -O binary test_program.elf test_program.bin
hexdump -v -e '1/4 "%08x\n"' test_program.bin > test_program.hex
```

Compare el contenido del archivo .hex generado automáticamente con los códigos de maquina entregados por **objdump**. ¿son iguales?